

- ① Doubt x
- ② Notes x

• Digital Notes

Module

- Async Programming
- Safety and validation
- Docker
- Langchain and Langgraph.
- Multi agents
- Crewai
 - Advanced Agent.
 - Web agents
 - Llamaindex Advanced RAG.
- Agentic and Graph.
- Hybrid search

demo/poc - 1/2
↓
Production.
Integrator.co AI
lok - openai
Scale.

Agenda.

- understanding Async
- How LLM use Async
 - use cases
 - Components

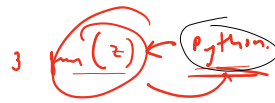
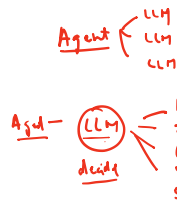
Async

vs

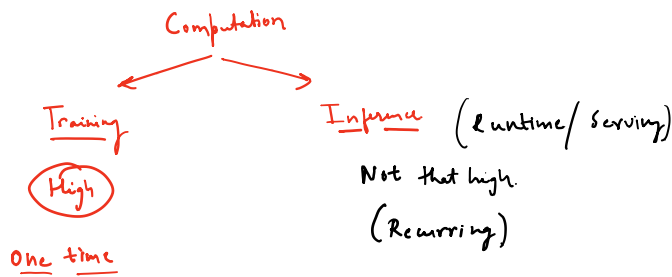
sync

LLM → Tool (Low computation)

↓
Algorithm (calc) → output
(high computation)

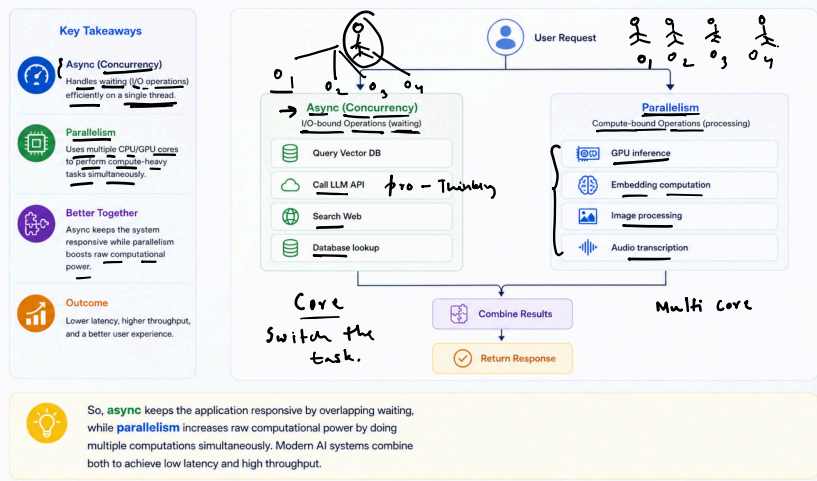


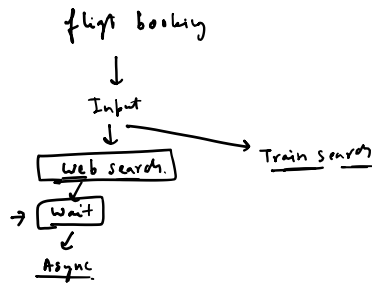
LLM — Decide — Tool — func(7) — Python



Real AI System Example

A production AI assistant often uses **both** at the same time:





3. async / await Fundamentals

The building blocks of asynchronous programming in Python.



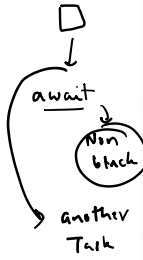
Task

cut

sleep(2)

async def f()

- print
- print
- print
- await ()
- pause

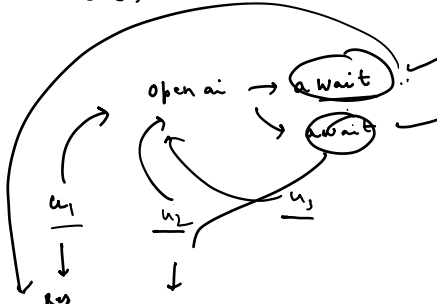


```

def _sanity_check():
    resp = client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "user", "content": "Reply with exactly one word: READY"}],
        max_tokens=5,
    )
    print("model replied:", resp.choices[0].message.content.strip())
  
```

SC() 1
SC()
SC()

Coroutine



Con {

```

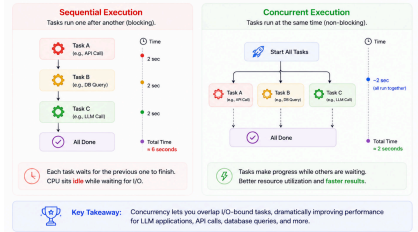
async def _sanity_check():
    resp = await client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "user", "content": "Reply with exactly one word: READY"}],
        max_tokens=5,
    )
    print("model replied:", resp.choices[0].message.content.strip())
    #task 2
    #task 3
    await _sanity_check()
  
```

X
Con
async

create task gather

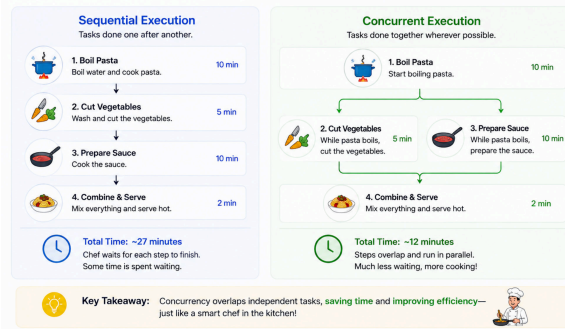
4. Sequential vs Concurrent Execution

Doing more in **less time** by running tasks together.



Sequential vs Concurrent Execution

Doing the same tasks in different ways. **Same goal, different time!**



Generator and yield

Async Generator.

streaming LLM responses

Grather.

Task - 300 → cancel
Timeout
error handling

9545126535

